

IT Equipment Request Handler

Week 7 Lab · MCP Server + ReAct Agent · Submission

Repo	github.com/kenmann01/it-service-mcp , branch feat/agent-react
Stack	Python 3.12+, MCP Python SDK v2 (stdio), Ollama qwen3:8b, uv, pytest
Policy	16-rule deterministic procedure in <code>evaluate_request</code> : approve / deny / escalate
Agent	Planner → TAO (Thought / Action / Observation) → Reflector, with explicit routing

1. Requirements doc and policy rules

[docs/requirements.md](#)

Equipment request policy

This document is the only source of truth for an equipment request. Decide each request by the procedure below. Do not add rules about monitors, refresh cycles, years between replacements, cost, or inventory. The decisions are `approve`, `deny`, and `escalate`. `approve` when the policy clearly covers the request. `deny` when the policy clearly does not cover it. `escalate` when the request is ambiguous or missing information, so a human decides.

Request

Every field is a string. A missing field, or a field that is empty or only spaces after trimming, is blank.

```
```json
{
 "employee": "name",
 "role": "role",
 "item_requested": "item",
 "reason": "the reason"
}
```

Before any rule, trim spaces and compare `role` and `item\_requested` in lowercase.

Allowed roles:

- `employee`
- `manager`
- `director` (this value means anyone above manager; the field must already say `director`)

Allowed items:

- `headphones`
- `phone`
- `laptop`

Any other role string or item string is not allowed. Do not map synonyms. `headset`, `mobile`, `laptop computer`, `vp`, and `executive` do not match.

## ## Reason class

Classify `reason` in lowercase. Use the first class that matches.

1. `week\_overlap` when the reason has all three of these:
  - `keep both` or `both laptops`
  - `week` or `7 days`
  - `return`
2. `unclear` when the reason has both a second-item signal and a first-item signal.
3. `second` when the reason has any second-item signal: `second`, `spare`, `additional`, `another`, `extra`, `already have`.
4. `first` when the reason has any first-item signal: `first`, `do not have`, `don't have`, `not have one yet`.
5. `replacement` when the reason has any replacement signal: `replacement`, `replace`, `replacing`, `broken`, `broke`, `lost`, `stolen`, `worn out`, `stopped working`.
6. `unclear` when none of the above match.

`week\_overlap` wins over `replacement`. A reason that says both "replacing" and "keep both for a week before returning" is `week\_overlap`.

## ## Decision procedure

Apply the first matching rule. Stop there.

1. Any required field is blank. Decision: `escalate`. Rule: `blank\_field`.
2. Role is not `employee`, `manager`, or `director`. Decision: `escalate`. Rule: `unknown\_role`.
3. Item is not `headphones`, `phone`, or `laptop`. Decision: `escalate`. Rule: `unknown\_item`.
4. Item is `laptop` and role is `employee`. Decision: `deny`. Rule: `employee\_laptop`. The policy does not cover employee laptops, whatever the reason says.
5. Item is `laptop`, role is `manager` or `director`, and the reason class is `week\_overlap`. Decision: `escalate`. Rule: `laptop\_week\_overlap`. Keeping both laptops for a week and then returning the old one to IT is a human decision.
6. Item is `laptop` and role is `manager` or `director`, for every other reason class. Decision: `deny`. Rule: `laptop\_not\_covered`. No laptop request is approved.
7. Item is `headphones`, role is `employee`, `manager`, or `director`, and the reason class is `replacement`. Decision: `approve`. Rule: `headphones\_replacement`.
8. Item is `headphones`, role is `employee`, `manager`, or `director`, and the reason class is `first`. Decision: `approve`. Rule: `headphones\_first`.
9. Item is `headphones` and the reason class is `second`. Decision: `deny`. Rule: `headphones\_second`. A spare pair while they keep the current ones is not approved.
10. Item is `phone`, role is `employee`, and the reason class is `replacement`. Decision: `approve`. Rule: `employee\_phone\_replacement`.
11. Item is `phone`, role is `employee`, and the reason class is `first`. Decision: `escalate`. Rule: `employee\_phone\_first`. An employee is approved only for a phone replacement; whether to grant a first company phone is a human decision.
12. Item is `phone`, role is `employee`, and the reason class is `second`. Decision: `deny`. Rule: `employee\_phone\_second`. An extra phone for an employee is not approved.
13. Item is `phone`, role is `manager` or `director`, and the reason class is `replacement`. Decision: `approve`. Rule: `manager\_phone\_replacement`.
14. Item is `phone`, role is `manager` or `director`, and the reason class is `first`. Decision: `approve`. Rule: `manager\_phone\_first`.
15. Item is `phone`, role is `manager` or `director`, and the reason class is `second`. Decision: `deny`. Rule: `manager\_phone\_second`. A second phone while they already have one is not approved.
16. Item is `headphones` or `phone` and the reason class is `unclear` or `week\_overlap`. Decision: `escalate`. Rule: `unclear\_reason`.

There is no later step. Steps 1 through 16 cover every request.

## ## Result

Return only this object:

```
```json
{
  "decision": "approve",
  "rule": "headphones_replacement"
}
```
```

`decision` is `approve`, `deny`, or `escalate`. `rule` is the rule id from the step that matched.

## [docs/policy.md](#)

### # Equipment request policy

This policy says which equipment requests are approved, which are denied, and which are sent to IT for a decision. It covers headphones, phones, and laptops.

To submit a request, give your name, your role, the item, and the reason. Roles are employee, manager, and director. Director means anyone above manager.

An approved request is issued under the rule below. A denied request is refused by this policy; the rule below says why. An escalated request is not approved and not refused by this policy. IT reviews it.

## ## Headphones

Every employee, manager, and director may request headphones.

- A replacement for the pair you have is approved.
- A first pair, when you do not have headphones yet, is approved.
- A second or spare pair, while you keep the pair you already have, is denied.

## ## Phones

Employees may request a replacement for a company phone they already have. That replacement is approved. Any other phone request from an employee is not approved: a first company phone is escalated, and an extra phone is denied.

Managers and directors may request one company phone.

- A replacement for the phone you have is approved.
- A first company phone, when you do not have one yet, is approved.
- A second phone, when you already have one, is denied.

## ## Laptops

A laptop request is denied. This policy does not approve laptops.

- A manager or director who asks to keep the new laptop and the old one for a week, and then return the old one to IT, is escalated. Keeping both laptops is a human decision.
- An employee who asks for a laptop is denied. This policy does not say what an employee may have.
- Any other laptop request from a manager or director is denied.

## ## Requests this policy does not cover

IT escalates a request when any of the following is true.

- Your name, role, item, or reason is missing.
- Your role is not employee, manager, or director.
- The item is not headphones, a phone, or a laptop.
- A headphones or phone request does not say whether it is a replacement, a first one, or a second one.

## 2. Minimal one-tool server and client connection

---

The simplest possible server (one trivial tool), then a client that connects, lists tools, and calls it. Captured terminal output below the sources.

[examples/minimal\\_server.py](#)

```
"""The simplest possible MCP server: one trivial tool, no business logic.

This is the suggested-approach step 1 artifact: prove the SDK installs, the
server starts, and the tool registers, before any real business logic exists.
"""

from mcp.server import MCPServer
from mcp.types import ToolAnnotations

mcp = MCPServer("Minimal Example")

_READ_ONLY = ToolAnnotations(read_only_hint=True)

def ping() -> str:
 """Return pong."""
 return "pong"

mcp.tool(annotations=_READ_ONLY)(ping)

if __name__ == "__main__":
 mcp.run()
```

[examples/minimal\\_client.py](#)

```
"""Minimal MCP client: connects to minimal_server.py, lists tools, calls one.

This is the suggested-approach step 3 artifact: proof that the plumbing works
end to end before the real tools are built.
"""

import sys
from pathlib import Path

import anyio
from mcp.client.session import ClientSession
from mcp.client.stdio import StdioServerParameters, stdio_client

SERVER = Path(__file__).resolve().parent / "minimal_server.py"

async def main() -> None:
 parameters = StdioServerParameters(command=sys.executable, args=[str(SERVER)])
 print(f"Connecting to {SERVER.name} over stdio...")
 async with stdio_client(parameters) as streams:
 async with ClientSession(streams[0], streams[1]) as session:
 await session.initialize()
 listed = await session.list_tools()
 names = [tool.name for tool in listed.tools]
 print(f"Registered tools: {names}")
 print("Calling ping()...")
 result = await session.call_tool("ping", {})
 print(f"ping() -> {result.content[0].text}")
```

```
if __name__ == "__main__":
 anyio.run(main)
```

[demo/outputs/minimal-server.txt](#)

```
Connecting to minimal_server.py over stdio...
Registered tools: ['ping']
Calling ping()...
ping() -> pong
```

### 3. MCP server code (all five tools)

---

[app/src/server.py](#)

```
from mcp.server import MCPServer
from mcp.types import ToolAnnotations

from tools.decision import evaluate_request
from tools.eligibility import check_request_eligibility
from tools.employee_info import get_employee_info
from tools.human_review import flag_for_human_review
from tools.policy_limits import get_policy_limits

mcp = MCPServer("Internal IT Service")

_READ_ONLY = ToolAnnotations(read_only_hint=True)
_ESCALATE = ToolAnnotations(read_only_hint=False, destructive_hint=False)

mcp.tool(annotations=_READ_ONLY)(get_employee_info)
mcp.tool(annotations=_READ_ONLY)(get_policy_limits)
mcp.tool(annotations=_READ_ONLY)(check_request_eligibility)
mcp.tool(annotations=_READ_ONLY)(evaluate_request)
mcp.tool(annotations=_ESCALATE)(flag_for_human_review)

if __name__ == "__main__":
 mcp.run()
```

[app/src/tools/employee\\_info.py](#)

```
"""Look up role, tenure, and equipment for one employee."""

from typing import NotRequired, TypedDict

from tools.data_store import load_employees

class EmployeeInfo(TypedDict):
 found: bool
 employee_id: str
 name: NotRequired[str]
 role: NotRequired[str]
 tenure_years: NotRequired[int]
 equipment: NotRequired[list[str]]

def get_employee_info(employee_id: str) -> EmployeeInfo:
 """Return role, tenure, and current equipment on file for an employee."""
 key = employee_id.strip()
 for employee in load_employees():
 if employee["employee_id"] == key:
 return {
 "found": True,
 "employee_id": employee["employee_id"],
 "name": employee["name"],
 "role": employee["role"],
 "tenure_years": employee["tenure_years"],
 "equipment": list(employee["equipment"]),
 }
 return {"found": False, "employee_id": key}
```

## app/src/tools/policy\_limits.py

```
"""Role eligibility and the period that covers each item."""

from typing import NotRequired, TypedDict

from tools.data_store import load_policy_limits

class ItemLimit(TypedDict):
 eligible: bool
 period: str

class PolicyLimits(TypedDict):
 role: str
 known: bool
 eligible_for: NotRequired[list[str]]
 items: NotRequired[dict[str, ItemLimit]]

def get_policy_limits(role: str) -> PolicyLimits:
 """Return what a role is eligible for and the period for each item."""
 key = role.strip().lower()
 limits = load_policy_limits()
 if key not in limits:
 return {"role": key, "known": False}
 items: dict[str, ItemLimit] = limits[key]
 eligible_for = [item for item, rule in items.items() if rule["eligible"]]
 return {
 "role": key,
 "known": True,
 "eligible_for": eligible_for,
 "items": items,
 }
```

## app/src/tools/eligibility.py

```
"""Whether a role may receive an item under at least one covered reason."""

from typing import TypedDict

from tools.employee_info import get_employee_info
from tools.policy_limits import get_policy_limits

class Eligibility(TypedDict):
 employee_id: str
 item: str
 eligible: bool

def check_request_eligibility(employee_id: str, item: str) -> Eligibility:
 """Return whether this employee's role may request this item under policy.

 True only when the role is covered for the item under at least one reason
 class. A laptop, an unknown item, or an unknown employee is not eligible.
 Equipment already on file is not consulted, and the reason text is not
 classified here.
 """
 info = get_employee_info(employee_id)
 normalized = item.strip().lower()
 if not info["found"]:
 return {"employee_id": info["employee_id"], "item": normalized, "eligible": False}
 limits = get_policy_limits(info["role"])
 items = limits.get("items") or {}
```

```

rule = items.get(normalized)
eligible = bool(limits["known"] and rule is not None and rule["eligible"])
return {"employee_id": info["employee_id"], "item": normalized, "eligible": eligible}

```

[app/src/tools/decision.py](#)

```

"""The 16-rule decision procedure from docs/requirements.md, as pure functions.

```

```

The request is consumed exactly as submitted: the claimed role is not
cross-checked against employee data, and equipment on file is not consulted.
"""

```

```

from typing import TypedDict

```

```

ROLES = ("employee", "manager", "director")
ITEMS = ("headphones", "phone", "laptop")

```

```

_WEEK_OVERLAP_SIGNALS = ("keep both", "both laptops")
_WEEK_SIGNALS = ("week", "7 days")
_RETURN_SIGNALS = ("return",)
_SECOND_SIGNALS = ("second", "spare", "additional", "another", "extra", "already have")
_FIRST_SIGNALS = ("first", "do not have", "don't have", "not have one yet")
_REPLACEMENT_SIGNALS = (
 "replacement",
 "replace",
 "replacing",
 "broken",
 "broke",
 "lost",
 "stolen",
 "worn out",
 "stopped working",
)

```

```

class Decision(TypedDict):
 decision: str
 rule: str

```

```

def classify_reason(reason: str) -> str:
 """Return the reason class: the first of the spec's classes that matches."""
 text = reason.strip().lower()
 overlap = any(s in text for s in _WEEK_OVERLAP_SIGNALS)
 week = any(s in text for s in _WEEK_SIGNALS)
 returns = any(s in text for s in _RETURN_SIGNALS)
 if overlap and week and returns:
 return "week_overlap"
 has_second = any(s in text for s in _SECOND_SIGNALS)
 has_first = any(s in text for s in _FIRST_SIGNALS)
 if has_second and has_first:
 return "unclear"
 if has_second:
 return "second"
 if has_first:
 return "first"
 if any(s in text for s in _REPLACEMENT_SIGNALS):
 return "replacement"
 return "unclear"

```

```

def _blank(*fields: str) -> bool:
 return any(not str(field).strip() for field in fields)

```

```

def evaluate_request(employee: str, role: str, item_requested: str, reason: str) -> Decision:

```

```

"""Apply the 16-rule first-match procedure to one request.

Returns {"decision": "approve"|"deny"|"escalate", "rule": "<id>"}. An
`escalate` decision means a human should decide; call flag_for_human_review.
"""
if _blank(employee, role, item_requested, reason):
 return {"decision": "escalate", "rule": "blank_field"}

normalized_role = role.strip().lower()
normalized_item = item_requested.strip().lower()
if normalized_role not in ROLES:
 return {"decision": "escalate", "rule": "unknown_role"}
if normalized_item not in ITEMS:
 return {"decision": "escalate", "rule": "unknown_item"}

reason_class = classify_reason(reason)
laptop = normalized_item == "laptop"
phone = normalized_item == "phone"
headphones = normalized_item == "headphones"
staff = normalized_role in ("manager", "director")

if laptop and normalized_role == "employee":
 return {"decision": "deny", "rule": "employee_laptop"}
if laptop and staff and reason_class == "week_overlap":
 return {"decision": "escalate", "rule": "laptop_week_overlap"}
if laptop:
 return {"decision": "deny", "rule": "laptop_not_covered"}

if headphones and reason_class == "replacement":
 return {"decision": "approve", "rule": "headphones_replacement"}
if headphones and reason_class == "first":
 return {"decision": "approve", "rule": "headphones_first"}
if headphones and reason_class == "second":
 return {"decision": "deny", "rule": "headphones_second"}

if phone and normalized_role == "employee" and reason_class == "replacement":
 return {"decision": "approve", "rule": "employee_phone_replacement"}
if phone and normalized_role == "employee" and reason_class == "first":
 return {"decision": "escalate", "rule": "employee_phone_first"}
if phone and normalized_role == "employee" and reason_class == "second":
 return {"decision": "deny", "rule": "employee_phone_second"}
if phone and staff and reason_class == "replacement":
 return {"decision": "approve", "rule": "manager_phone_replacement"}
if phone and staff and reason_class == "first":
 return {"decision": "approve", "rule": "manager_phone_first"}
if phone and staff and reason_class == "second":
 return {"decision": "deny", "rule": "manager_phone_second"}

Rule 16: headphones or phone with an unclear or week_overlap reason class.
return {"decision": "escalate", "rule": "unclear_reason"}

```

[app/src/tools/human\\_review.py](#)

```

"""Process-local queue of requests escalated for a person to review."""

from typing import TypedDict

_reviews: list["ReviewRecord"] = []

class ReviewRecord(TypedDict):
 review_id: str
 employee_id: str
 request: str
 reason: str
 status: str

```

```

def flag_for_human_review(employee_id: str, request: str, reason: str) -> ReviewRecord:
 """Escalate a request for human review. This records the request and never approves it."""
 record: ReviewRecord = {
 "review_id": f"R{len(_reviews) + 1:03d}",
 "employee_id": employee_id,
 "request": request,
 "reason": reason,
 "status": "escalated",
 }
 _reviews.append(record)
 return record

def pending_reviews() -> list[ReviewRecord]:
 return list(_reviews)

def clear_reviews() -> None:
 _reviews.clear()

```

[app/src/tools/data\\_store.py](#)

```

"""Load the synthetic employee and policy JSON used by the IT tools."""

import json
from functools import lru_cache
from pathlib import Path
from typing import Any

DATA_DIR = Path(__file__).resolve().parents[2] / "data"

@lru_cache(maxsize=1)
def load_employees() -> tuple[dict[str, Any], ...]:
 payload = json.loads((DATA_DIR / "employees.json").read_text(encoding="utf-8"))
 return tuple(payload["employees"])

@lru_cache(maxsize=1)
def load_policy_limits() -> dict[str, Any]:
 return json.loads((DATA_DIR / "policy_limits.json").read_text(encoding="utf-8"))

```

## 4. Unit tests and passing run

app/tests/test\_tools.py

```
"""Isolated checks of the IT tools against the JSON mock data. No MCP transport."""

import sys
import unittest
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from tools.eligibility import check_request_eligibility
from tools.employee_info import get_employee_info
from tools.human_review import clear_reviews, flag_for_human_review
from tools.policy_limits import get_policy_limits

class EmployeeInfoTests(unittest.TestCase):
 def test_known_employee(self) -> None:
 info = get_employee_info("E001")
 self.assertTrue(info["found"])
 self.assertEqual(info["name"], "Grace Hopper")
 self.assertEqual(info["role"], "employee")
 self.assertEqual(info["tenure_years"], 14)
 self.assertEqual(info["equipment"], ["headphones", "laptop"])

 def test_strips_id_whitespace(self) -> None:
 info = get_employee_info(" E010 ")
 self.assertTrue(info["found"])
 self.assertEqual(info["name"], "Linus Torvalds")
 self.assertEqual(info["role"], "employee")

 def test_unknown_employee(self) -> None:
 info = get_employee_info("E999")
 self.assertEqual(info, {"found": False, "employee_id": "E999"})

class PolicyLimitTests(unittest.TestCase):
 def test_each_role(self) -> None:
 for role in ("employee", "manager", "director"):
 limits = get_policy_limits(role)
 self.assertTrue(limits["known"])
 self.assertEqual(limits["eligible_for"], ["headphones", "phone"])
 self.assertFalse(limits["items"]["laptop"]["eligible"])
 self.assertIn("period", limits["items"]["headphones"])

 def test_employee_phone_period_is_replacement_only(self) -> None:
 limits = get_policy_limits(" Employee ")
 self.assertEqual(limits["role"], "employee")
 self.assertIn("replacement", limits["items"]["phone"]["period"].lower())
 self.assertNotIn("year", limits["items"]["phone"]["period"].lower())

 def test_manager_phone_covers_a_first_phone(self) -> None:
 period = get_policy_limits("manager")["items"]["phone"]["period"].lower()
 self.assertIn("first", period)
 self.assertIn("second", period)

 def test_unknown_role(self) -> None:
 limits = get_policy_limits("executive")
 self.assertEqual(limits, {"role": "executive", "known": False})

class EligibilityTests(unittest.TestCase):
 def test_headphones_are_eligible_for_every_role(self) -> None:
```

```

 for employee_id in ("E001", "E003", "E004"):
 result = check_request_eligibility(employee_id, "Headphones ")
 self.assertTrue(result["eligible"])
 self.assertEqual(result["item"], "headphones")

 def test_phone_is_eligible_for_every_role(self) -> None:
 for employee_id in ("E005", "E007", "E013"):
 result = check_request_eligibility(employee_id, "phone")
 self.assertTrue(result["eligible"])

 def test_laptop_is_never_eligible(self) -> None:
 for employee_id in ("E010", "E009", "E008"):
 result = check_request_eligibility(employee_id, "laptop")
 self.assertFalse(result["eligible"])

 def test_unknown_item_and_unknown_employee(self) -> None:
 self.assertFalse(check_request_eligibility("E001", "monitor")["eligible"])
 self.assertFalse(check_request_eligibility("E999", "headphones")["eligible"])

 def test_does_not_use_equipment_on_file(self) -> None:
 without_headphones = check_request_eligibility("E011", "headphones")
 with_headphones = check_request_eligibility("E001", "headphones")
 self.assertTrue(without_headphones["eligible"])
 self.assertTrue(with_headphones["eligible"])

class HumanReviewTests(unittest.TestCase):
 def setUp(self) -> None:
 clear_reviews()

 def test_two_escalations_get_distinct_ids(self) -> None:
 first = flag_for_human_review("E010", "laptop", "I need a laptop for my work.")
 second = flag_for_human_review("E004", "headphones", "Second pair.")
 self.assertEqual(first["status"], "escalated")
 self.assertEqual(second["status"], "escalated")
 self.assertNotEqual(first["review_id"], second["review_id"])
 self.assertEqual(first["employee_id"], "E010")
 self.assertEqual(second["request"], "headphones")

 def test_blank_fields_are_still_escalated(self) -> None:
 record = flag_for_human_review(" ", " ", " ")
 self.assertEqual(record["status"], "escalated")
 self.assertEqual(record["request"], " ")

if __name__ == "__main__":
 unittest.main()

```

## app/tests/test\_decision.py

```

"""Checks of the 16-rule decision procedure and the reason classifier."""

import json
import sys
import unittest
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from tools.decision import classify_reason, evaluate_request

FIXTURES = Path(__file__).resolve().parents[2] / "fixtures" / "mock-requests.json"

class ClassifyReasonTests(unittest.TestCase):
 def test_each_signal_class(self) -> None:

```

```

 self.assertEqual(classify_reason("Second pair, I already have one."), "second")
 self.assertEqual(classify_reason("First pair, I do not have headphones."), "first")
 self.assertEqual(classify_reason("Replacement of my broken headphones."), "replacement")

 def test_week_overlap_needs_all_three_signals(self) -> None:
 self.assertEqual(
 classify_reason("Keep both for a week, then return the old one."), "week_overlap"
)
 self.assertEqual(classify_reason("Keep both laptops, they are great."), "unclear")
 self.assertEqual(classify_reason("Return it next week."), "unclear")

 def test_week_overlap_beats_replacement(self) -> None:
 reason = "Replacing my laptop. I will keep both for a week before I return the old one."
 self.assertEqual(classify_reason(reason), "week_overlap")

 def test_second_plus_first_is_unclear(self) -> None:
 self.assertEqual(classify_reason("A second pair since I do not have one."), "unclear")

 def test_unrecognized_reason_is_unclear(self) -> None:
 self.assertEqual(classify_reason("Protein goals."), "unclear")

 def test_matching_ignores_case_and_outer_spaces(self) -> None:
 self.assertEqual(classify_reason(" It BROKE. Need a REPLACEMENT. "), "replacement")

class EvaluateRequestTests(unittest.TestCase):
 def test_blank_field_escalates(self) -> None:
 self.assertEqual(
 evaluate_request("Grace Hopper", "employee", "headphones", " "),
 {"decision": "escalate", "rule": "blank_field"},
)
 self.assertEqual(
 evaluate_request(" ", "employee", "headphones", "broken"),
 {"decision": "escalate", "rule": "blank_field"},
)

 def test_unknown_role_escalates(self) -> None:
 self.assertEqual(
 evaluate_request("Mani", "FDE", "vanilla latte", "protein goals"),
 {"decision": "escalate", "rule": "unknown_role"},
)
 self.assertEqual(
 evaluate_request("Mani", "executive", "headphones", "replacement"),
 {"decision": "escalate", "rule": "unknown_role"},
)

 def test_unknown_item_escalates(self) -> None:
 self.assertEqual(
 evaluate_request("Grace Hopper", "employee", "monitor", "replacement"),
 {"decision": "escalate", "rule": "unknown_item"},
)

 def test_employee_laptop_is_denied(self) -> None:
 self.assertEqual(
 evaluate_request("Linus Torvalds", "employee", "laptop", "I need it for work."),
 {"decision": "deny", "rule": "employee_laptop"},
)

 def test_manager_laptop_week_overlap_escalates(self) -> None:
 reason = "Replacing my laptop. I will keep both for a week before returning the old one."
 self.assertEqual(
 evaluate_request("Ada Lovelace", "manager", "laptop", reason),
 {"decision": "escalate", "rule": "laptop_week_overlap"},
)

 def test_other_manager_laptops_are_denied(self) -> None:
 self.assertEqual(

```

```

 evaluate_request("Ada Lovelace", "director", "laptop", "broken screen"),
 {"decision": "deny", "rule": "laptop_not_covered"},
)

def test_headphones_replacement_and_first_are_approved(self) -> None:
 self.assertEqual(
 evaluate_request("Grace Hopper", "employee", "headphones", "broken"),
 {"decision": "approve", "rule": "headphones_replacement"},
)
 self.assertEqual(
 evaluate_request("Alan Turing", "manager", "headphones", "first pair"),
 {"decision": "approve", "rule": "headphones_first"},
)

def test_headphones_second_is_denied(self) -> None:
 self.assertEqual(
 evaluate_request("Margaret Hamilton", "director", "headphones", "spare pair"),
 {"decision": "deny", "rule": "headphones_second"},
)

def test_employee_phone_rules(self) -> None:
 self.assertEqual(
 evaluate_request("Radia Perlman", "employee", "phone", "my phone broke"),
 {"decision": "approve", "rule": "employee_phone_replacement"},
)
 self.assertEqual(
 evaluate_request("Radia Perlman", "employee", "phone", "first company phone"),
 {"decision": "escalate", "rule": "employee_phone_first"},
)
 self.assertEqual(
 evaluate_request("Tim Berners-Lee", "employee", "phone", "additional phone"),
 {"decision": "deny", "rule": "employee_phone_second"},
)

def test_manager_and_director_phone_rules(self) -> None:
 self.assertEqual(
 evaluate_request("Barbara Liskov", "manager", "phone", "replacement, it stopped working"),
 {"decision": "approve", "rule": "manager_phone_replacement"},
)
 self.assertEqual(
 evaluate_request("Barbara Liskov", "manager", "phone", "I do not have one yet"),
 {"decision": "approve", "rule": "manager_phone_first"},
)
 self.assertEqual(
 evaluate_request("Donald Knuth", "director", "phone", "another phone"),
 {"decision": "deny", "rule": "manager_phone_second"},
)

def test_unclear_and_week_overlap_reasons_escalate(self) -> None:
 self.assertEqual(
 evaluate_request("Grace Hopper", "employee", "headphones", "not sure what I need"),
 {"decision": "escalate", "rule": "unclear_reason"},
)
 reason = "Replacing my phone. I will keep both for a week and return the old one."
 self.assertEqual(
 evaluate_request("Ada Lovelace", "manager", "phone", reason),
 {"decision": "escalate", "rule": "unclear_reason"},
)

def test_normalizes_role_item_and_reason(self) -> None:
 self.assertEqual(
 evaluate_request(" Grace Hopper ", " Employee ", " Headphones ", " It BROKE. "),
 {"decision": "approve", "rule": "headphones_replacement"},
)

```

```

class FixtureTests(unittest.TestCase):

```

```

def test_all_fixture_cases_match_expected_decision(self) -> None:
 with FIXTURES.open(encoding="utf-8") as handle:
 cases = json.load(handle)["cases"]
 self.assertEqual(len(cases), 10)
 for case in cases:
 with self.subTest(case_id=case["id"]):
 request = case["request"]
 result = evaluate_request(
 request["employee"],
 request["role"],
 request["item_requested"],
 request["reason"],
)
 self.assertEqual(result["decision"], case["expected"], case["id"])

if __name__ == "__main__":
 unittest.main()

```

[app/tests/test\\_react.py](#)

```

"""End-to-end ReAct loop checks with a scripted LLM and scripted tools.

No Ollama and no live server: the loop is exercised through the same seams
production uses, which is what CI runs.
"""

import sys
import unittest
from itertools import repeat
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from agent.llm_client import LLMResponse, ToolCall
from agent.react import FALLBACK_RESPONSE, run_agent
from agent.trace import Trace

class FakeLLM:
 """Scripted chat(): returns prepared LLMResponse objects in order."""

 def __init__(self, script) -> None:
 self.script = iter(script)
 self.messages: list[list[dict]] = []

 def chat(self, messages, tools=None) -> LLMResponse:
 self.messages.append(messages)
 return next(self.script)

class FakeMCP:
 """Scripted tools: returns one prepared result per tool name, records calls."""

 def __init__(self, responses: dict) -> None:
 self.responses = responses
 self.calls: list[tuple[str, dict]] = []

 async def list_tool_schemas(self) -> list[dict]:
 return [{"type": "function", "function": {"name": name, "parameters": {}}} for name in self.responses]

 async def call_tool(self, name: str, arguments: dict):
 self.calls.append((name, arguments))
 return self.responses[name]

```

```

def _responses(decision: str, rule: str) -> dict:
 return {
 "get_employee_info": {
 "found": True,
 "employee_id": "E001",
 "name": "Grace Hopper",
 "role": "employee",
 "tenure_years": 14,
 "equipment": ["headphones", "laptop"],
 },
 "evaluate_request": {"decision": decision, "rule": rule},
 "flag_for_human_review": {"review_id": "R001", "status": "escalated"},
 }

```

```

def _call(name: str, content: str, arguments: dict) -> LLMResponse:
 return LLMResponse(content=content, tool_call=ToolCall(id="call_1", name=name, arguments=arguments))

```

```

APPROVE_REQUEST = "Hi, I am E001 Grace Hopper. My headphones broke; I need a replacement."

```

```

class ApprovePathTests(unittest.TestCase):
 def test_approve_routes_to_a_draft_without_escalation(self) -> None:
 mcp = FakeMCP(_responses("approve", "headphones_replacement"))
 llm = FakeLLM(
 [
 _call("get_employee_info", "Finding the requester.", {"employee_id": "E001"}),
 _call(
 "evaluate_request",
 "Applying the policy.",
 {
 "employee": "Grace Hopper",
 "role": "employee",
 "item_requested": "headphones",
 "reason": "broken",
 },
),
 LLMResponse(content="Grace, your replacement headphones are approved."),
 LLMResponse(content="CONFIRMED"),
]
)

 result = run_agent(APPROVE_REQUEST, llm=llm, mcp=mcp)

 self.assertEqual(result.final_response, "Grace, your replacement headphones are approved.")
 self.assertEqual(result.decision, "approve")
 self.assertEqual(result.rule, "headphones_replacement")
 self.assertIsNone(result.review_id)
 self.assertEqual(
 [name for name, _ in mcp.calls], ["get_employee_info", "evaluate_request"]
)

 def test_trace_records_tao_and_route_steps(self) -> None:
 mcp = FakeMCP(_responses("approve", "headphones_replacement"))
 llm = FakeLLM(
 [
 _call("get_employee_info", "Finding the requester.", {"employee_id": "E001"}),
 _call("evaluate_request", "Applying the policy.", {}),
 LLMResponse(content="Approved."),
 LLMResponse(content="CONFIRMED"),
]
)

 result = run_agent(APPROVE_REQUEST, llm=llm, mcp=mcp)

 types = [step.type for step in result.trace.steps]

```

```

 for expected in ("thought", "action", "observation", "route", "draft", "reflection"):
 self.assertIn(expected, types)
 route = next(step for step in result.trace.steps if step.type == "route")
 self.assertIn("approve", route.content)

class EscalatePathTests(unittest.TestCase):
 def test_escalation_flags_for_human_review(self) -> None:
 mcp = FakeMCP(_responses("escalate", "laptop_week_overlap"))
 llm = FakeLLM(
 [
 _call("get_employee_info", "Finding the requester.", {"employee_id": "E005"}),
 _call("evaluate_request", "Applying the policy.", {}),
 _call(
 "flag_for_human_review",
 "This needs a human.",
 {"employee_id": "E005", "request": "laptop", "reason": "week overlap"},
),
 LLMResponse(content="Escalated to human review as R001."),
 LLMResponse(content="CONFIRMED"),
]
)

 result = run_agent("Manager laptop request.", llm=llm, mcp=mcp)

 self.assertEqual(result.decision, "escalate")
 self.assertEqual(result.rule, "laptop_week_overlap")
 self.assertEqual(result.review_id, "R001")
 self.assertEqual(
 [name for name, _ in mcp.calls][-1], "flag_for_human_review"
)

 def test_deny_never_calls_the_escalation_tool(self) -> None:
 mcp = FakeMCP(_responses("deny", "employee_laptop"))
 llm = FakeLLM(
 [
 _call("get_employee_info", "Finding the requester.", {"employee_id": "E010"}),
 _call("evaluate_request", "Applying the policy.", {}),
 LLMResponse(content="Laptops are not covered for employees."),
 LLMResponse(content="CONFIRMED"),
]
)

 result = run_agent("Employee laptop request.", llm=llm, mcp=mcp)

 self.assertEqual(result.decision, "deny")
 self.assertEqual(result.rule, "employee_laptop")
 self.assertNotIn("flag_for_human_review", [name for name, _ in mcp.calls])

class LoopGuardTests(unittest.TestCase):
 def test_max_iterations_returns_the_fallback(self) -> None:
 mcp = FakeMCP(_responses("approve", "headphones_replacement"))
 llm = FakeLLM(
 repeat(
 _call("evaluate_request", "Applying the policy.", {}),
)
)

 result = run_agent(APPROVE_REQUEST, llm=llm, mcp=mcp, max_iterations=3)

 self.assertEqual(result.final_response, FALLBACK_RESPONSE)
 self.assertEqual(result.decision, "approve")
 self.assertTrue(any(step.type == "error" for step in result.trace.steps))

 def test_draft_without_policy_check_is_bounced(self) -> None:
 mcp = FakeMCP(_responses("approve", "headphones_replacement"))

```

```

llm = FakeLLM(
 [
 LLMResponse(content="I would approve this."),
 _call("evaluate_request", "Checking the policy first.", {}),
 LLMResponse(content="Grace, your replacement headphones are approved."),
 LLMResponse(content="CONFIRMED"),
]
)

result = run_agent(APPROVE_REQUEST, llm=llm, mcp=mcp)

self.assertEqual(result.decision, "approve")
self.assertTrue(
 any("bouncing" in step.content for step in result.trace.steps)
)
self.assertEqual(
 [name for name, _ in mcp.calls][0], "evaluate_request"
)

class ReflectionSeamTests(unittest.TestCase):
 def test_reflector_can_correct_the_draft(self) -> None:
 mcp = FakeMCP(_responses("approve", "headphones_replacement"))
 llm = FakeLLM(
 [
 _call("get_employee_info", "Finding the requester.", {"employee_id": "E001"}),
 _call("evaluate_request", "Applying the policy.", {}),
 LLMResponse(content="Draft v1."),
 LLMResponse(content="CORRECTED: Grace, your replacement headphones are approved under rule
headphones_replacement."),
]
)

 result = run_agent(APPROVE_REQUEST, llm=llm, mcp=mcp)

 self.assertTrue(
 result.final_response.startswith("Grace, your replacement headphones")
)
 self.assertTrue(
 any("corrected" in step.content.lower() for step in result.trace.steps)
)

 def test_trace_can_run_silently(self) -> None:
 trace = Trace(echo=False)
 trace.add("thought", "hidden")
 self.assertEqual(trace.to_text(), "[THOUGHT] hidden")

if __name__ == "__main__":
 unittest.main()

```

[app/tests/test\\_reflection.py](#)

```

"""Checks of the Reflector's CONFIRMED / CORRECTED handling, with a fake LLM."""

import sys
import unittest
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from agent.llm_client import LLMResponse
from agent.reflection import reflect
from agent.trace import Trace

```

```

class FakeLLM:
 """Scripted chat(): returns prepared LLMResponse objects in order."""

 def __init__(self, script: list[LLMResponse]) -> None:
 self.script = iter(script)
 self.messages: list[list[dict]] = []

 def chat(self, messages, tools=None) -> LLMResponse:
 self.messages.append(messages)
 return next(self.script)

 def _trace_with_observation() -> Trace:
 trace = Trace(echo=False)
 trace.add("observation", '{"decision": "approve", "rule": "headphones_replacement"}')
 return trace

class ReflectTests(unittest.TestCase):
 def test_confirmed_keeps_the_draft(self) -> None:
 llm = FakeLLM([LLMResponse(content="CONFIRMED")])
 trace = _trace_with_observation()
 result = reflect("Your replacement headphones are approved.", trace, llm)
 self.assertTrue(result.confirmed)
 self.assertEqual(result.final_draft, "Your replacement headphones are approved.")

 def test_prompt_includes_draft_and_observations(self) -> None:
 llm = FakeLLM([LLMResponse(content="CONFIRMED")])
 trace = _trace_with_observation()
 reflect("Draft text.", trace, llm)
 prompt = llm.messages[0][-1]["content"]
 self.assertIn("Draft text.", prompt)
 self.assertIn("headphones_replacement", prompt)

 def test_corrected_replaces_the_draft(self) -> None:
 llm = FakeLLM([LLMResponse(content="CORRECTED: The corrected draft.")])
 result = reflect("The draft.", _trace_with_observation(), llm)
 self.assertFalse(result.confirmed)
 self.assertEqual(result.final_draft, "The corrected draft.")
 self.assertEqual(result.original, "The draft.")

 def test_empty_correction_keeps_the_original(self) -> None:
 llm = FakeLLM([LLMResponse(content="CORRECTED:")])
 result = reflect("The draft.", _trace_with_observation(), llm)
 self.assertEqual(result.final_draft, "The draft.")

 def test_unparseable_reply_fails_open(self) -> None:
 llm = FakeLLM([LLMResponse(content="The draft looks fine to me.")])
 result = reflect("The draft.", _trace_with_observation(), llm)
 self.assertTrue(result.confirmed)
 self.assertEqual(result.final_draft, "The draft.")

if __name__ == "__main__":
 unittest.main()

```

[demo/outputs/pytest.txt](#)

```

===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Genspark\it-service-
mcp\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Genspark\it-service-mcp
configfile: pyproject.toml
testpaths: app/tests
plugins: anyio-4.15.1

```

collecting ... collected 45 items

```
app/tests/test_decision.py::ClassifyReasonTests::test_each_signal_class PASSED [2%]
app/tests/test_decision.py::ClassifyReasonTests::test_matching_ignores_case_and_outer_spaces PASSED [4%]
app/tests/test_decision.py::ClassifyReasonTests::test_second_plus_first_is_unclear PASSED [6%]
app/tests/test_decision.py::ClassifyReasonTests::test_unrecognized_reason_is_unclear PASSED [8%]
app/tests/test_decision.py::ClassifyReasonTests::test_week_overlap_beats_replacement PASSED [11%]
app/tests/test_decision.py::ClassifyReasonTests::test_week_overlap_needs_all_three_signals PASSED [13%]
app/tests/test_decision.py::EvaluateRequestTests::test_blank_field_escalates PASSED [15%]
app/tests/test_decision.py::EvaluateRequestTests::test_employee_laptop_is_denied PASSED [17%]
app/tests/test_decision.py::EvaluateRequestTests::test_employee_phone_rules PASSED [20%]
app/tests/test_decision.py::EvaluateRequestTests::test_headphones_replacement_and_first_are_approved PASSED [22%]
app/tests/test_decision.py::EvaluateRequestTests::test_headphones_second_is_denied PASSED [24%]
app/tests/test_decision.py::EvaluateRequestTests::test_manager_and_director_phone_rules PASSED [26%]
app/tests/test_decision.py::EvaluateRequestTests::test_manager_laptop_week_overlap_escalates PASSED [28%]
app/tests/test_decision.py::EvaluateRequestTests::test_normalizes_role_item_and_reason PASSED [31%]
app/tests/test_decision.py::EvaluateRequestTests::test_other_manager_laptops_are_denied PASSED [33%]
app/tests/test_decision.py::EvaluateRequestTests::test_unclear_and_week_overlap_reasons_escalate PASSED [35%]
app/tests/test_decision.py::EvaluateRequestTests::test_unknown_item_escalates PASSED [37%]
app/tests/test_decision.py::EvaluateRequestTests::test_unknown_role_escalates PASSED [40%]
app/tests/test_decision.py::FixtureTests::test_all_fixture_cases_match_expected_decision
app/tests/test_decision.py::FixtureTests::test_all_fixture_cases_match_expected_decision PASSED [42%]
app/tests/test_react.py::ApprovePathTests::test_approve_routes_to_a_draft_without_escalation PASSED [44%]
app/tests/test_react.py::ApprovePathTests::test_trace_records_tao_and_route_steps PASSED [46%]
app/tests/test_react.py::EscalatePathTests::test_deny_never_calls_the_escalation_tool PASSED [48%]
app/tests/test_react.py::EscalatePathTests::test_escalation_flags_for_human_review PASSED [51%]
app/tests/test_react.py::LoopGuardTests::test_max_iterations_returns_the_fallback PASSED [53%]
app/tests/test_react.py::ReflectionSeamTests::test_reflector_can_correct_the_draft PASSED [55%]
app/tests/test_react.py::ReflectionSeamTests::test_trace_can_run_silently PASSED [57%]
app/tests/test_reflection.py::ReflectTests::test_confirmed_keeps_the_draft PASSED [60%]
app/tests/test_reflection.py::ReflectTests::test_corrected_replaces_the_draft PASSED [62%]
app/tests/test_reflection.py::ReflectTests::test_empty_correction_keeps_the_original PASSED [64%]
app/tests/test_reflection.py::ReflectTests::test_prompt_includes_draft_and_observations PASSED [66%]
app/tests/test_reflection.py::ReflectTests::test_unparseable_reply_fails_open PASSED [68%]
app/tests/test_tools.py::EmployeeInfoTests::test_known_employee PASSED [71%]
app/tests/test_tools.py::EmployeeInfoTests::test_strips_id_whitespace PASSED [73%]
app/tests/test_tools.py::EmployeeInfoTests::test_unknown_employee PASSED [75%]
app/tests/test_tools.py::PolicyLimitTests::test_each_role PASSED [77%]
app/tests/test_tools.py::PolicyLimitTests::test_employee_phone_period_is_replacement_only PASSED [80%]
app/tests/test_tools.py::PolicyLimitTests::test_manager_phone_covers_a_first_phone PASSED [82%]
app/tests/test_tools.py::PolicyLimitTests::test_unknown_role PASSED [84%]
app/tests/test_tools.py::EligibilityTests::test_does_not_use_equipment_on_file PASSED [86%]
app/tests/test_tools.py::EligibilityTests::test_headphones_are_eligible_for_every_role PASSED [88%]
app/tests/test_tools.py::EligibilityTests::test_laptop_is_never_eligible PASSED [91%]
app/tests/test_tools.py::EligibilityTests::test_phone_is_eligible_for_every_role PASSED [93%]
app/tests/test_tools.py::EligibilityTests::test_unknown_item_and_unknown_employee PASSED [95%]
app/tests/test_tools.py::HumanReviewTests::test_blank_fields_are_still_escalated PASSED [97%]
app/tests/test_tools.py::HumanReviewTests::test_two_escalations_get_distinct_ids PASSED [100%]
```

===== 45 passed, 10 subtests passed in 0.71s =====

## 5. Agent code, including the MCP client connection

[app/src/agent/react.py](#)

```
"""Planner -> TAO -> Reflector, the whiteboard pipeline.
```

```
Planner: the system prompt below; the model states its plan before acting.
```

```
TAO: the Thought/Action/Observation loop over the MCP server's tools.
```

```
Routing: an explicit [ROUTE] step after evaluate_request returns a decision.
```

```
Reflector: agent.reflection.reflect confirms or corrects the final draft.
```

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
from dataclasses import dataclass
```

```
from typing import Any
```

```
import anyio
```

```
from agent.llm_client import OllamaClient
```

```
from agent.reflection import reflect
```

```
from agent.trace import Trace
```

```
from mcp_client import MCPClient
```

```
from tools.data_store import load_employees
```

```
MAX_ITERATIONS = 8
```

```
_ROUTES = {
```

```
 "approve": "draft the approval, naming the rule",
```

```
 "deny": "draft the refusal, naming the rule; do not call flag_for_human_review",
```

```
 "escalate": "call flag_for_human_review, then draft the handoff with the review id",
```

```
}
```

```
def _roster() -> str:
```

```
 lines = []
```

```
 for info in load_employees():
```

```
 lines.append(f"- {info['employee_id']}: {info['name']} ({info['role']})")
```

```
 return "\n".join(lines)
```

```
def _build_system_prompt() -> str:
```

```
 return f"""You are the IT equipment request agent for the Internal IT Service MCP server.
```

```
REQUEST
```

```
The user submits an equipment request in natural language. Extract four fields:
```

```
employee (the person's name), role, item_requested (headphones, phone, or laptop),
```

```
and reason.
```

```
ROSTER
```

```
{_roster()}
```

```
When the request names a person without an id, find their id above. When it gives
```

```
an id, use it as given.
```

```
TOOLS
```

```
- get_employee_info(employee_id): the employee's role, tenure, and equipment on file. Call this first to establish who is asking and what their role is.
```

```
- get_policy_limits(role): what a role is eligible for and how often.
```

```
- check_request_eligibility(employee_id, item): static coverage check for a role and item.
```

```
- evaluate_request(employee, role, item_requested, reason): applies the full written policy and returns two fields, decision (approve, deny, or escalate) and rule (the rule id). This tool decides the request; never decide policy yourself.
```

```
- flag_for_human_review(employee_id, request, reason): side-effecting escalation to the human review queue.
```

#### PLAN

1. Call `get_employee_info` to identify the requester **and** their role.
2. Call `evaluate_request` **with** the four extracted fields. Use the role returned by `get_employee_info`, **not** one you infer, unless `get_employee_info` does **not** find the requester: then use the role exactly **as** the user stated it, unusual **as** it may be. If a field **is** missing **from** the request, **pass** an empty string **for** it. Pass the reason text **as** the user wrote it.
3. Route by the decision `evaluate_request` returned: `{json.dumps(_ROUTES)}`
4. Write the final draft **and** nothing **else**. The draft **is** the message that goes to the requester: an approval says what **is** granted **and** cites the rule; a denial says it **is** refused **and** cites the rule **and** the policy reason; a handoff names the review id **and** why. Keep it under six sentences **and** address the requester by name. Never mention tools, iterations, routing, **or** drafting inside the draft.

Before every tool call, write one short sentence explaining why. EVERY request **is** decided by `evaluate_request`: even when the item **or** role looks unrelated to IT equipment, extract the four fields, call `evaluate_request`, **and** route by its decision. Only an escalate decision goes to `flag_for_human_review`. Never decide policy yourself, never invent policy rules, **and** never answer a request without calling `evaluate_request` first."""

```
SYSTEM_PROMPT = _build_system_prompt()
```

```
FALLBACK_RESPONSE = (
 "Unable to reach a decision after multiple attempts. "
 "Please try again or contact IT support."
)
```

```
@dataclass
```

```
class AgentResult:
```

```
 request: str
```

```
 final_response: str
```

```
 trace: Trace
```

```
 decision: str | None = None
```

```
 rule: str | None = None
```

```
 review_id: str | None = None
```

```
def run_agent(
 request: str,
 llm: Any = None,
 mcp: Any = None,
 max_iterations: int = MAX_ITERATIONS,
) -> AgentResult:
```

```
 """Run one request through the pipeline.
```

```
 """
```

```
 """
```

```
 llm and mcp are seams for tests: pass a FakeLLM and a FakeMCP to run the
```

```
 loop without Ollama or a live server. Production callers pass neither.
```

```
 """
```

```
 llm = llm or OllamaClient.from_env()
```

```
 if mcp is not None:
```

```
 return anyio.run(_react_loop, request, llm, mcp, max_iterations)
```

```
 return anyio.run(_react_loop_live, request, llm, max_iterations)
```

```
async def _react_loop_live(request: str, llm: Any, max_iterations: int) -> AgentResult:
```

```
 async with MCPClient() as server:
```

```
 return await _react_loop(request, llm, server, max_iterations)
```

```
async def _react_loop(
 request: str, llm: Any, mcp: Any, max_iterations: int
) -> AgentResult:
```

```
 trace = Trace()
```

```
 tools = await mcp.list_tool_schemas()
```

```
 messages: list[dict[str, Any]] = [
 {"role": "system", "content": SYSTEM_PROMPT},
```

```

 {"role": "user", "content": request},
]
 decision: str | None = None
 rule: str | None = None
 review_id: str | None = None
 policy_checked = False

 for iteration in range(1, max_iterations + 1):
 response = llm.chat(messages, tools=tools)

 if response.has_tool_call and response.tool_call:
 action = response.tool_call
 thought = response.content or f"I need to call {action.name}."
 trace.add("thought", f"(iteration {iteration}) {thought}")
 trace.add(
 "action", f"{action.name}({json.dumps(action.arguments, sort_keys=True)})"
)
 try:
 observation = await mcp.call_tool(action.name, action.arguments)
 except (RuntimeError, json.JSONDecodeError) as error:
 observation = {"error": str(error)}
 trace.add("observation", json.dumps(observation, sort_keys=True, default=str))

 if action.name == "evaluate_request" and isinstance(observation, dict):
 policy_checked = True
 decision = observation.get("decision")
 rule = observation.get("rule")
 if decision:
 trace.add("route", f"{decision} ({rule}) -> {_ROUTES.get(decision, 'unknown route')}")
 if action.name == "flag_for_human_review" and isinstance(observation, dict):
 review_id = observation.get("review_id")

 messages.append(
 {"role": "assistant", "content": thought, "tool_calls": [action.to_dict()]}
)
 messages.append(
 {
 "role": "tool",
 "content": json.dumps(observation, sort_keys=True, default=str),
 "tool_call_id": action.id,
 }
)
 continue

 draft = (response.content or "").strip()
 if not draft:
 trace.add("error", f"Iteration {iteration}: empty model reply; retrying.")
 continue

 if not policy_checked:
 # Guard: a draft may not exist before the decision tool has run.
 trace.add("error", "Draft attempted before evaluate_request ran; bouncing back.")
 messages.append({"role": "assistant", "content": draft})
 messages.append(
 {
 "role": "user",
 "content": "Before responding, call evaluate_request with the four "
 "request fields and route by its decision. Call the tool now.",
 }
)
 continue

 trace.add("draft", draft)
 reflection_result = reflect(draft, trace, llm)
 trace.add("reflection", reflection_result.summary)
 return AgentResult(
 request=request,

```

```

 final_response=reflection_result.final_draft,
 trace=trace,
 decision=decision,
 rule=rule,
 review_id=review_id,
)

 trace.add("error", f"Max iterations ({max_iterations}) reached without a final answer.")
 return AgentResult(
 request=request,
 final_response=FALLBACK_RESPONSE,
 trace=trace,
 decision=decision,
 rule=rule,
 review_id=review_id,
)

```

[app/src/mcp\\_client.py](#)

```

"""Stdio MCP client shared by the hand-check script and, later, the planner and reflector."""

```

```

import json
import sys
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Any

import anyio
from mcp.client.session import ClientSession
from mcp.client.stdio import StdioServerParameters, stdio_client

SERVER_PATH = Path(__file__).resolve().parent / "server.py"

def list_tools() -> list[str]:
 """Launch the IT server and return the registered tool names."""
 return anyio.run(_list_tools)

def call_tool(name: str, arguments: dict[str, Any]) -> Any:
 """Launch the IT server, call one tool, and return its JSON result."""
 return anyio.run(_call_tool, name, arguments)

async def _list_tools() -> list[str]:
 async with _session() as session:
 listed = await session.list_tools()
 return [tool.name for tool in listed.tools]

async def _call_tool(name: str, arguments: dict[str, Any]) -> Any:
 async with _session() as session:
 result = await session.call_tool(name, arguments)
 if getattr(result, "is_error", False):
 raise RuntimeError(_error_text(result))
 structured = getattr(result, "structured_content", None)
 if structured is not None:
 return structured
 return json.loads(_error_text(result))

def _error_text(result: Any) -> str:
 blocks = getattr(result, "content", None) or []
 parts: list[str] = []
 for block in blocks:
 text = getattr(block, "text", None)

```

```

 if text:
 parts.append(text)
 return "\n".join(parts) if parts else "tool call failed"

@asynccontextmanager
async def _session(server_path: Path | None = None):
 parameters = StdioServerParameters(
 command=sys.executable,
 args=[str(server_path or SERVER_PATH)],
)
 async with stdio_client(parameters) as streams:
 async with ClientSession(streams[0], streams[1]) as session:
 await session.initialize()
 yield session

class MCPClient:
 """One persistent stdio server session for a whole agent run.

 The TAO loop calls several tools per request; keeping the session open
 avoids relaunching the server for every call. Tool schemas come back in
 OpenAI/Ollama function format, ready to pass to the LLM.
 """

 def __init__(self, server_path: Path | None = None) -> None:
 self.server_path = server_path or SERVER_PATH

 async def __aenter__(self) -> "MCPClient":
 self._session_cm = _session(self.server_path)
 self._session = await self._session_cm.__aenter__()
 return self

 async def __aexit__(self, *exc_info: object) -> None:
 await self._session_cm.__aexit__(*exc_info)

 async def list_tool_schemas(self) -> list[dict[str, Any]]:
 listed = await self._session.list_tools()
 return [
 {
 "type": "function",
 "function": {
 "name": tool.name,
 "description": tool.description or "",
 "parameters": tool.input_schema,
 },
 }
 for tool in listed.tools
]

 async def call_tool(self, name: str, arguments: dict[str, Any]) -> Any:
 result = await self._session.call_tool(name, arguments)
 if getattr(result, "is_error", False):
 raise RuntimeError(_error_text(result))
 structured = getattr(result, "structured_content", None)
 if structured is not None:
 return structured
 text = _error_text(result)
 return json.loads(text) if text else None

```

[app/src/agent/llm\\_client.py](#)

```

"""Ollama chat client with OpenAI-style tool calling, used by the TAO loop."""

from __future__ import annotations

```

```

import os
import re
from dataclasses import dataclass
from typing import Any

import httpx

DEFAULT_HOST = "http://localhost:11434"
DEFAULT_MODEL = "qwen3:8b"

_THINK_RE = re.compile(r"<think>.*?</think>", re.DOTALL)

def strip_think(text: str) -> str:
 """Drop qwen3 <think> blocks so reasoning tokens never leak into drafts."""
 text = _THINK_RE.sub("", text)
 if "</think>" in text:
 text = text.split("</think>", 1)[1]
 return text.strip()

@dataclass
class ToolCall:
 id: str
 name: str
 arguments: dict[str, Any]

 def to_dict(self) -> dict[str, Any]:
 return {
 "id": self.id,
 "type": "function",
 "function": {"name": self.name, "arguments": self.arguments},
 }

@dataclass
class LLMResponse:
 content: str | None
 tool_call: ToolCall | None = None

 @property
 def has_tool_call(self) -> bool:
 return self.tool_call is not None

class OllamaClient:
 """Synchronous /api/chat client; parses tool_calls when the model emits them."""

 def __init__(
 self,
 host: str = DEFAULT_HOST,
 model: str = DEFAULT_MODEL,
 temperature: float = 0.3,
 timeout: float = 240.0,
 think: bool = False,
 keep_alive: str = "30m",
) -> None:
 self.host = host.rstrip("/")
 self.model = model
 self.temperature = temperature
 self.timeout = timeout # generous: a cold start loads the model into VRAM
 self.think = think
 self.keep_alive = keep_alive

 @classmethod
 def from_env(cls) -> "OllamaClient":
 return cls(

```

```

 host=os.environ.get("OLLAMA_HOST", DEFAULT_HOST),
 model=os.environ.get("OLLAMA_MODEL", DEFAULT_MODEL),
)

 def chat(
 self, messages: list[dict[str, Any]], tools: list[dict[str, Any]] | None = None
) -> LLMResponse:
 payload: dict[str, Any] = {
 "model": self.model,
 "messages": messages,
 "stream": False,
 "think": self.think,
 "keep_alive": self.keep_alive,
 "options": {"temperature": self.temperature},
 }
 if tools:
 payload["tools"] = tools

 response = httpx.post(f"{self.host}/api/chat", json=payload, timeout=self.timeout)
 response.raise_for_status()
 message = response.json().get("message", {})
 content = strip_think(message.get("content") or "")

 calls = message.get("tool_calls") or []
 if calls:
 call = calls[0]
 function = call.get("function", {})
 return LLMResponse(
 content=content,
 tool_call=ToolCall(
 id=call.get("id") or "call_1",
 name=function.get("name", ""),
 arguments=function.get("arguments") or {},
),
)
 return LLMResponse(content=content)

```

[app/src/agent/reflection.py](#)

```

"""Reflector: a second LLM pass that confirms or corrects the draft."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Any

from agent.trace import Trace

REFLECTION_PROMPT = """You are a policy compliance reviewer for IT equipment requests.

DRAFT RESPONSE:
{draft}

TOOL OBSERVATIONS (from the ReAct trace):
{observations}

Check the draft against the observations:
1. Is the draft an actual message to the requester? If it is a note about drafting, or mentions tools,
iterations, or routing, that is a failure: correct it into a proper message.
2. Does it claim anything the observations do not support?
3. Does it state the decision and rule exactly as the evaluate_request tool returned them?
4. If the request was escalated, does the handoff mention the review id and the real reason?
5. Does it commit to anything (approval, refusal) the data does not clearly support?

If the draft is accurate, reply exactly: CONFIRMED
If the draft needs fixing, reply exactly: CORRECTED: <the revised draft>

```

```

"""

@dataclass
class ReflectionResult:
 confirmed: bool
 final_draft: str
 original: str
 summary: str

def reflect(draft: str, trace: Trace, llm: Any) -> ReflectionResult:
 """Review the draft against the trace's observations; fail open to confirmed."""
 prompt = REFLECTION_PROMPT.format(draft=draft, observations=trace.observations() or "(none)")
 response = llm.chat([{"role": "user", "content": prompt}])
 text = (response.content or "").strip()

 if text.startswith("CORRECTED:"):
 revised = text.removeprefix("CORRECTED:").strip() or draft
 return ReflectionResult(False, revised, draft, f"Draft corrected: {revised}")
 if text.startswith("CONFIRMED"):
 return ReflectionResult(True, draft, draft, "Draft confirmed against tool observations")
 return ReflectionResult(True, draft, draft, "Reflection reply was not parseable; draft kept")

```

[app/src/agent/trace.py](#)

```

"""Trace of Thought / Action / Observation steps.

Every step is printed as it happens so a terminal capture doubles as the
submission screenshot, and stored structurally so the Reflector can review
the observations.
"""

from dataclasses import dataclass, field

_LABELS = {
 "thought": "[THOUGHT]",
 "action": "[ACTION]",
 "observation": "[OBSERVATION]",
 "route": "[ROUTE]",
 "draft": "[DRAFT]",
 "reflection": "[REFLECTION]",
 "error": "[ERROR]",
}

@dataclass
class TraceStep:
 type: str
 content: str

@dataclass
class Trace:
 echo: bool = True
 steps: list[TraceStep] = field(default_factory=list)

 def add(self, type: str, content: str) -> TraceStep:
 step = TraceStep(type, content)
 self.steps.append(step)
 if self.echo:
 label = _LABELS.get(type, f"[{type.upper()}]")
 prefix = "\n" if type in ("thought", "draft") else ""
 print(f"{prefix}{label} {content}", flush=True)
 return step

```

```

def observations(self) -> str:
 return "\n".join(step.content for step in self.steps if step.type == "observation")

def to_text(self) -> str:
 return "\n".join(
 f"[_LABELS.get(step.type, step.type.upper())] {step.content}" for step in self.steps
)

```

#### app/scripts/agent.py

```

"""Host app: run one natural-language request through the agent.

From the repo root:
 uv run python app/scripts/agent.py "I need a replacement, my headphones broke"
 uv run python app/scripts/agent.py "<request>" --model qwen3:8b
"""

import argparse
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from agent.llm_client import OllamaClient
from agent.react import run_agent

def main() -> None:
 parser = argparse.ArgumentParser(
 description="IT equipment request agent (Planner -> TAO -> Reflector)"
)
 parser.add_argument("request", help="natural-language equipment request")
 parser.add_argument("--host", default=None, help="Ollama host (default OLLAMA_HOST)")
 parser.add_argument("--model", default=None, help="Ollama model (default OLLAMA_MODEL)")
 args = parser.parse_args()

 env_client = OllamaClient.from_env()
 llm = OllamaClient(host=args.host or env_client.host, model=args.model or env_client.model)

 result = run_agent(args.request, llm=llm)

 print()
 print("=== FINAL RESPONSE ===")
 print(result.final_response)
 summary = f"=== decision={result.decision} rule={result.rule}"
 if result.review_id:
 summary += f" review_id={result.review_id}"
 print(summary + " ===")

if __name__ == "__main__":
 main()

```

## 6. Full ReAct traces for the 4 test requests

Full live traces (Thought → Action → Observation → Route → Draft → Reflection) for the four test requests, run against gwen3:8b and the MCP server.

**Test request:** "Hi, I'm Grace Hopper (E001). My headphones broke, so I need a replacement."

[demo/outputs/demo-approve.txt](#)

```
[THOUGHT] (iteration 1) I need to call get_employee_info.
[ACTION] get_employee_info({"employee_id": "E001"})
[OBSERVATION] {"employee_id": "E001", "equipment": ["headphones", "laptop"], "found": true, "name": "Grace Hopper", "role": "employee", "tenure_years": 14}

[THOUGHT] (iteration 2) I need to call evaluate_request.
[ACTION] evaluate_request({"employee": "Grace Hopper", "item_requested": "headphones", "reason": "My headphones broke, so I need a replacement.", "role": "employee"})
[OBSERVATION] {"decision": "approve", "rule": "headphones_replacement"}
[ROUTE] approve (headphones_replacement) -> draft the approval, naming the rule

[DRAFT] Your request for a replacement headphones has been approved under rule headphones_replacement, Grace Hopper. The replacement will be processed shortly. Let me know if you need anything else.
[REFLECTION] Draft corrected: Your request for a replacement headphones has been approved under rule headphones_replacement. The replacement will be processed shortly. Let me know if you need anything else.

=== FINAL RESPONSE ===
Your request for a replacement headphones has been approved under rule headphones_replacement. The replacement will be processed shortly. Let me know if you need anything else.
=== decision=approve rule=headphones_replacement ===
```

**Test request:** "Hi, I'm Linus Torvalds (E010). I need a laptop for my work."

[demo/outputs/demo-deny.txt](#)

```
[THOUGHT] (iteration 1) I need to call get_employee_info.
[ACTION] get_employee_info({"employee_id": "E010"})
[OBSERVATION] {"employee_id": "E010", "equipment": ["laptop"], "found": true, "name": "Linus Torvalds", "role": "employee", "tenure_years": 10}

[THOUGHT] (iteration 2) I need to call evaluate_request.
[ACTION] evaluate_request({"employee": "Linus Torvalds", "item_requested": "laptop", "reason": "for my work", "role": "employee"})
[OBSERVATION] {"decision": "deny", "rule": "employee_laptop"}
[ROUTE] deny (employee_laptop) -> draft the refusal, naming the rule; do not call flag_for_human_review

[DRAFT] I need to draft the denial message.
[REFLECTION] Draft corrected: Your request for a laptop has been denied. The rule "employee_laptop" does not support granting laptops to employees. Please consult your manager for further assistance.

=== FINAL RESPONSE ===
Your request for a laptop has been denied. The rule "employee_laptop" does not support granting laptops to employees. Please consult your manager for further assistance.
=== decision=deny rule=employee_laptop ===
```

**Test request:** "Hi, I'm Ada Lovelace (E009). I'm replacing my laptop and will keep both for a week before returning the old one to IT."

[demo/outputs/demo-escalate-week-overlap.txt](#)

```
[THOUGHT] (iteration 1) I need to call get_employee_info.
[ACTION] get_employee_info({"employee_id": "E009"})
[OBSERVATION] {"employee_id": "E009", "equipment": ["phone", "laptop"], "found": true, "name": "Ada Lovelace", "role": "manager", "tenure_years": 6}
```

```
[THOUGHT] (iteration 2) I need to call evaluate_request.
[ACTION] evaluate_request({"employee": "Ada Lovelace", "item_requested": "laptop", "reason": "I'm replacing my
laptop and will keep both for a week before returning the old one to IT.", "role": "manager"})
[OBSERVATION] {"decision": "escalate", "rule": "laptop_week_overlap"}
[ROUTE] escalate (laptop_week_overlap) -> call flag_for_human_review, then draft the handoff with the review
id

[THOUGHT] (iteration 3) I need to call flag_for_human_review.
[ACTION] flag_for_human_review({"employee_id": "E009", "reason": "Replacing laptop and keeping both for a week
before returning the old one to IT", "request": "laptop"})
[OBSERVATION] {"employee_id": "E009", "reason": "Replacing laptop and keeping both for a week before returning
the old one to IT", "request": "laptop", "review_id": "R001", "status": "escalated"}

[DRAFT] Your request for a laptop has been escalated for human review. The review ID is R001. We will notify
you once a decision is made. Thank you for your understanding, Ada Lovelace.
[REFLECTION] Draft corrected: Your request for a laptop has been escalated for human review. The review ID is
R001. The reason for escalation is that you are replacing your laptop and will keep both for a week before
returning the old one to IT. We will notify you once a decision is made. Thank you for your understanding, Ada
Lovelace.

=== FINAL RESPONSE ===
Your request for a laptop has been escalated for human review. The review ID is R001. The reason for
escalation is that you are replacing your laptop and will keep both for a week before returning the old one to
IT. We will notify you once a decision is made. Thank you for your understanding, Ada Lovelace.
=== decision=escalate rule=laptop_week_overlap review_id=R001 ===
```

**Test request:** "Hi, I'm Radia Perlman (E005). I need a second phone since I don't have one right now."

[demo/outputs/demo-escalate-unclear.txt](#)

```
[THOUGHT] (iteration 1) I need to call get_employee_info.
[ACTION] get_employee_info({"employee_id": "E005"})
[OBSERVATION] {"employee_id": "E005", "equipment": ["phone", "laptop"], "found": true, "name": "Radia
Perlman", "role": "employee", "tenure_years": 9}

[THOUGHT] (iteration 2) I need to call evaluate_request.
[ACTION] evaluate_request({"employee": "Radia Perlman", "item_requested": "phone", "reason": "I need a second
phone since I don't have one right now.", "role": "employee"})
[OBSERVATION] {"decision": "escalate", "rule": "unclear_reason"}
[ROUTE] escalate (unclear_reason) -> call flag_for_human_review, then draft the handoff with the review id

[THOUGHT] (iteration 3) I need to call flag_for_human_review.
[ACTION] flag_for_human_review({"employee_id": "E005", "reason": "I need a second phone since I don't have one
right now.", "request": "second phone"})
[OBSERVATION] {"employee_id": "E005", "reason": "I need a second phone since I don't have one right now.",
"request": "second phone", "review_id": "R001", "status": "escalated"}

[DRAFT] Your request for a second phone has been escalated for human review. The review ID is R001. A human
will assess your request and get back to you shortly. Thank you for your patience, Radia.
[REFLECTION] Draft corrected: Your request for a second phone has been escalated for human review. The review
ID is R001. A human will assess your request and get back to you shortly. Thank you for your patience, Radia.

=== FINAL RESPONSE ===
Your request for a second phone has been escalated for human review. The review ID is R001. A human will
assess your request and get back to you shortly. Thank you for your patience, Radia.
=== decision=escalate rule=unclear_reason review_id=R001 ===
```

## 7. Final decision / response for each of the 4 requests

---

The final response returned to the requester for each of the four test requests, with the routed decision, rule id, and review id where applicable.

### approve request

Your request for a replacement headphones has been approved under rule headphones\_replacement. The replacement will be processed shortly. Let me know if you need anything else.  
=== decision=approve rule=headphones\_replacement ===

### deny request

Your request for a laptop has been denied. The rule "employee\_laptop" does not support granting laptops to employees. Please consult your manager for further assistance.  
=== decision=deny rule=employee\_laptop ===

### week-overlap request

Your request for a laptop has been escalated for human review. The review ID is R001. The reason for escalation is that you are replacing your laptop and will keep both for a week before returning the old one to IT. We will notify you once a decision is made. Thank you for your understanding, Ada Lovelace.  
=== decision=escalate rule=laptop\_week\_overlap review\_id=R001 ===

### unclear request

Your request for a second phone has been escalated for human review. The review ID is R001. A human will assess your request and get back to you shortly. Thank you for your patience, Radia.  
=== decision=escalate rule=unclear\_reason review\_id=R001 ===

## 8. Evidence of the reflection step

---

**deny request** — Draft was a meta-note; the Reflector rewrote it into a real refusal:

[DRAFT]I need to draft the denial message.

[REFLECTION] Draft corrected: Your request for a laptop has been denied. The rule "employee\_laptop" does not support granting laptops to employees. Please consult your manager for further assistance.

**week-overlap request** — Draft omitted the escalation reason; the Reflector added it:

[DRAFT]Your request for a laptop has been escalated for human review. The review ID is R001. We will notify you once a decision is made. Thank you for your understanding, Ada Lovelace.

[REFLECTION] Draft corrected: Your request for a laptop has been escalated for human review. The review ID is R001. The reason for escalation is that you are replacing your laptop and will keep both for a week before returning the old one to IT. We will notify you once a decision is made. Thank you for your understanding, Ada Lovelace.

## 9. Passing pipeline run

demo/outputs/ci-run.txt

```
✓ master ci · 36968555626
Triggered via push less than a minute ago

JOBS
✓ test in 10s (ID 110717504019)

ANNOTATIONS
! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24:
actions/checkout@v4, astral-sh/setup-uv@v5. For more information see: https://github.blog/changelog/2025-09-
19-deprecation-of-node-20-on-github-actions-runners/
test: .github#2

- "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see
https://github.com/actions/runner-images/issues/14748"
test: .github#1

For more information about the job, try: gh run view --job=110717504019
View this run on GitHub: https://github.com/kenmann01/it-service-mcp/actions/runs/36968555626
```

demo/outputs/pool-results.txt

```
RESULTS (24 cases)
=====
id expected got rule status sec

approve-01-headphones-replacement approve approve headphones_replacement OK 13
approve-02-headphones-first approve approve headphones_first OK 13
approve-03-employee-phone-replacement approve approve employee_phone_replacement OK 14
approve-04-manager-headphones-replacement approve approve headphones_replacement OK 16
approve-05-manager-phone-first approve approve manager_phone_first OK 14
approve-06-director-phone-replacement approve approve manager_phone_replacement OK 14
deny-01-employee-laptop deny deny employee_laptop OK 14
deny-02-employee-phone-second deny deny employee_phone_second OK 17
deny-03-headphones-spare deny deny headphones_second OK 15
deny-04-director-phone-second deny deny manager_phone_second OK 14
deny-05-manager-laptop deny deny laptop_not_covered OK 15
deny-06-headphones-extra deny deny headphones_second OK 14
escalate-1-01-unknown-role escalate escalate blank_field OK 18
escalate-1-02-blank-reason escalate escalate blank_field OK 19
escalate-1-03-unclear-contradiction escalate escalate unclear_reason OK 20
escalate-1-04-employee-phone-first escalate escalate employee_phone_first OK 20
escalate-1-05-unknown-item escalate escalate unknown_item OK 19
escalate-1-06-no-signal escalate escalate unclear_reason OK 20
escalate-2-01-manager-laptop-week-overlap escalate escalate laptop_week_overlap OK 20
escalate-2-02-manager-laptop-7-days escalate escalate laptop_week_overlap OK 21
escalate-2-03-director-laptop-week-overlap escalate escalate laptop_week_overlap OK 20
escalate-2-04-manager-phone-week-overlap escalate escalate unclear_reason OK 19
escalate-2-05-headphones-week-overlap escalate escalate unclear_reason OK 21
escalate-2-06-director-laptop-handback escalate escalate laptop_week_overlap OK 21

matched expectations: 24/24
clear_approve: 6/6 OK
clear_deny: 6/6 OK
escalate_ambiguous: 6/6 OK
escalate_week_overlap: 6/6 OK
```